Lecture 2: Deep Learning

Andrei A Simion

Columbia University

September 6, 2025

Outline

- Softmax and Cross-Entropy
- The XOR Problem
- Feed Forward Neural Networks
- Forward and Back Propagation
- Solving the XOR Problem

Outline

- Softmax and Cross-Entropy
- The XOR Problem
- Feed Forward Neural Networks
- Forward and Back Propagation
- Solving the XOR Problem

Softmax

- Whenever we have a vector of values, we might be interested in the maximum one: Which index attains the maximum?
- For example if $z = (z_1, \ldots, z_K)$ then perhaps z_3 is largest
- The softmax is a "soft" approximation of the max, where we turn each component of z into a probability
- Specifically, this is done by saying

$$a = softmax(z) = \left(\frac{e^{z_1}}{\sum_{j=1}^{K} e^{z_j}}, \ldots, \frac{e^{z_K}}{\sum_{j=1}^{K} e^{z_j}}\right)$$

Softmax

- The *softmax* function turns each value into a probability
- The maximum component of the vector a is the same as the maximum component of $a = softmax(z)$
- We have

$$argmax_j z_j = argmax_j a_j$$

- But, unlike the max which is nondifferentiable, the *softmax* is differentiable
- We have

$$\frac{dsoftmax}{dz_k}(z) = softmax(z)_k(1 - softmax(z)_k) = \frac{e^{z_k}(\sum_{j=1, j\neq k}^{d} e^{z_j})}{(\sum_{j=1}^{K} e^{z_j})^2}$$

- Softmax is a generalization of $\sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z}$
  - If we have $z = (z_1, z_2)$ then

$$softmax(z) = (\frac{e^{z_1}}{e^{z_1} + e^{z_2}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2}}) = (\sigma(z_1 - z_2), 1 - \sigma(z_1 - z_2))$$

  - I.e. When you have two classes, the σ function is enough

Cross Entropy and Softmax in General

- When we have a model that takes x and outputs a label y, we might have that y is in a class with more than 2 values
- If for a fixed $x^{(i)}$ the model outputs a set of probabilities per class given by $a^{(i)} = (a_1^{(i)}, \ldots, a_K^{(i)})$ when a typical loss to consider is

$$\ell(\theta) = -\frac{\sum_{i=1}^{N} \sum_{k=1}^{K} oneHot(y^{(i)})_k \log a_k^{(i)}}{N}$$

  - Here, the $oneHot(y^{(i)}) \in \mathbb{R}^K$ is the one-hot representation of the ith response value: a vector of $1/0$ values where only one value is 1 and all others are zero; $oneHot(y^{(i)})_{y^{(i)}} = 1$
  - Here, $a^{(i)}$ is a vector of probabilities and $\sum_{k=1}^{K} a_k^{(i)} = 1$ and $a_k^{(i)} \geq 0 \; \forall k$
- A good model is such that if the true label is at index k so $oneHot(y)_k = 1$ then $a_k \sim 1$ and all others probabilities are small
- But how do we get a model taking in $x^{(i)}$ to output a vector $a^{(i)}$ which is a probability?

Cross Entropy - Review of how this works in practice

- Typically, a model outputs a set of logits $z \in \mathbb{R}^K$ which are real values
- After, we apply a softmax to get probabilities $a = softmax(z)$
- Then, these values define a loss ℓ which we minimize
  - Note that for a training point with $oneHot(y^{(i)})_k = 1$ the above reduces to $-\log a_k^{(i)}$
  - Since $0 \leq a_k^{(i)} \leq 1$ a good model will be such that it's parameters are adjusted so that $a_k^{(i)} \sim 1$
- When we optimize ℓ, the model parameters are adjusted so that on average this behavior happens on the Training set $\{(x^{(i)}, y^{(i)})\}_{i=1}^N$
  - If we do this too much on the training set, then we overfit and need to deal with this problem

Cross Entropy - σ vs Softmax again

- Final note: If we have 2 classes, we can use Binary or regular Cross Entropy
  - Model outputs $a = softmax(z) \in \mathbb{R}^2$ then we use the one-hot of $oneHot(y) \in \mathbb{R}^2$ and consider minimizing

$$\ell = -(oneHot(y)_1 \log a_1 + oneHot(y)_2 \log a_2)$$

  - Model outputs $a = \sigma(z) \in \mathbb{R}$ then we use $y \in \mathbb{R} = 1/0$ and consider minimizing

$$\ell = -(y \log \sigma(z) + (1 - y) \log 1 - \sigma(z))$$

- These are the same thing!

Outline

- Softmax and Cross-Entropy
- The XOR Problem
- Feed Forward Neural Networks
- Forward and Back Propagation
- Solving the XOR Problem

XOR: Why we need Neural Networks

- We can make Logistic Regression models where we take two features x_1 and x_2 and define a new feature $x_3 = x_1 * x_2$ and try and estimate $y \sim \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$
- The above model is still a Logistic Regression and linear in its parameters; there are still things basic (Convex) models like Logistic Regression cannot do
  - When the loss is convex (i.e. $\ell(\theta) \sim \theta^2$) and we minimize $\ell(\theta)$ we know that any local optima is global (Theorem), so these type of problems are "easy"
  - There are many algorithms to optimize convex problems, and these algorithms have nice guarantees ...
- However, we want to look here in detail at one example where Logistic Regression cannot do well

XOR: The Problem

- Consider two variables x_1, x_2 that take on values of 1 or 0
- Set $y = x_1 + x_2 - 2x_1x_2$
- The y is known as XOR and the values are as follows
  - If $x_1 = 1$ and $x_2 = 1$ then $y = 0$
  - If $x_1 = 1$ and $x_2 = 0$ then $y = 1$
  - If $x_1 = 0$ and $x_2 = 1$ then $y = 1$
  - If $x_1 = 0$ and $x_2 = 0$ then $y = 0$
- Problem: Given 4 data points $\{(x^{(i)}, y^{(i)})\}_{i=1}^{4}$ that represent the 4 points above, can we find a classifier that can correctly classify these points?
- At a high level, we need a model that has

$$p(y^{(i)}|x^{(i)}) > 1/2 \; \forall i$$

- For example, we want $p(y = 1|x_1 = 1, x_2 = 0) > 1/2$

XOR: Let's try Logistic Regression

- Let's try a linear classifier like Logistic Regression
- What do we need? Set $\theta \in \mathbb{R}^2$ and $\beta \in \mathbb{R}$
- We need to find $\theta \in \mathbb{R}^2$ and $\beta \in \mathbb{R}$ so that

$$p(y^{(i)}|x^{(i)}) > 1/2 \ \forall i$$

- We consider $z = \theta^\intercal x + \beta \in \mathbb{R}$
- $a = \sigma(z) \in \mathbb{R}$ is the probability that the point should be classified as 1
  - What do we need? For example when $x = (1, 0)$ we know $y = 1$
  - In this case, we want $\sigma(z)$ to be more than $\frac{1}{2}$ since we want the probability of being 1 to be high
  - On the other hand, if $x = (1, 1)$ we know $y = 0$ so then we want $\sigma(z)$ to less than $\frac{1}{2}$, since we want the probability of 0 (which is $1 - \sigma(z)$) to be high

XOR: Getting restrictions on the parameters

- Consider the case $x = (1, 0)$ for which we know $y = 1$
- We have $\theta^\intercal x + \beta = \theta_1 + \beta$ and we want $\sigma(\theta_1 + \beta) > \frac{1}{2}$
- This is the same as $\frac{1}{1+e^{-(\theta_1+\beta)}} > \frac{1}{2}$
- Cross multiplying, this is

$$2 > 1 + e^{-(\theta_1+\beta)}$$

- We get $e^{(\theta_1+\beta)} > 1$
- Simplifying, this gives

$$\theta_1 + \beta > 0$$

- Similarly, if we consider other cases we get
  - If $x = (0, 1)$ then $y = 1$ so $\theta_2 + \beta > 0$
  - If $x = (1, 1)$ then $y = 0$ so $\theta_1 + \theta_2 + \beta < 0$
  - If $x = (0, 0)$ then $y = 0$ so $\beta < 0$

XOR - No Linear Classifier

- So we have
  - If $x = (1, 0)$, $y = 1$ we need $\sigma(\theta_1 + \beta) > 1/2$ so

$$\theta_1 + \beta > 0$$

  - If $x = (0, 1)$, $y = 1$ we need $\sigma(\theta_2 + \beta) > 1/2$ so

$$\theta_2 + \beta > 0$$

  - If $x = (1, 1)$, $y = 0$ we need $1 - \sigma(\theta_1 + \theta_2 + \beta) > 1/2$ so

$$\theta_1 + \theta_2 + \beta < 0$$

  - If $x = (0, 0)$, $y = 0$ we need $1 - \sigma(\beta) > 1/2$ so

$$\beta < 0$$

XOR - No Linear Classifier

- But, if we add the first two inequalities we get

$$\theta_1 + \theta_2 + 2\beta > 0$$

- Moving a β over and using the last inequality gives

$$\theta_1 + \theta_2 + \beta > -\beta > 0$$

- However, this contradicts the third inequality above
- Upshot: We can't find a linear classifier to do this, we need more!

XOR: What about more parameters? Maybe softmax?

- Consider $y = 1, 2$ here as the response where $0 \to 1$ and $1 \to 2$ from before
  - Basically, we shift the rule up by 1 so $y = x_1 + x_2 - 2x_1x_2 + 1$
- Consider the case when we use a Softmax as the final layer
- In this case, when we try $\Theta \in \mathbb{R}^{2\times 2}$ and $\beta \in \mathbb{R}^2$
- Assume Θ has columns θ_1 and θ_2
  - $\theta_1 = (\Theta_{11}, \Theta_{21})$ and $\theta_2 = (\Theta_{12}, \Theta_{22})$
  - This model has $a \in \mathbb{R}^2$ that is 2 dimensional; but, as we saw, this is equivalent to dimension 1 since $a_1 + a_2 = 1$
- We consider $z = \Theta^\intercal x + \beta \in \mathbb{R}^2$
- We then set $a = softmax(z) \in \mathbb{R}^2$ so that

$$a_y = p(y|x) = \frac{e^{\theta_y^\intercal x + \beta_y}}{\sum_{y'} e^{\theta_{y'}^\intercal x + \beta_{y'}}}$$

  - Here, we have $\{\theta_y\}_{y=1}^2 \in \mathbb{R}^2$ and $\beta \in \mathbb{R}^2$
  - These are parameters we apply to x before the $softmax(z)$
  - Crucially, we have not applied any nonlinearity to z

XOR - More parameters?

- To solve XOR, we want $p(y^{(i)}|x^{(i)}) > 1/2 \; \forall \; i$
- Consider the case when $y = 2$ and let's see what happens when $x = (1, 0)$
- We need

$$p(y = 2|x = (1, 0)) = \frac{e^{\theta_2^\intercal x + \beta_2}}{\sum_{y'} e^{\theta_{y'}^\intercal x + \beta_{y'}}} > \frac{1}{2}$$

- This is

$$\frac{e^{\theta_2^\intercal x + \beta_2}}{e^{\theta_1^\intercal x + \beta_1} + e^{\theta_2^\intercal x + \beta_2}} > \frac{1}{2}$$

- This is the same as

$$\frac{1}{1 + e^{-[(\theta_2^\intercal - \theta_1^\intercal)x + (\beta_2 - \beta_1)]}} > \frac{1}{2}$$

- Let $\gamma = \theta_2 - \theta_1 \in \mathbb{R}^2$ and $\nu = \beta_2 - \beta_1 \in \mathbb{R}^1$
- It is easy to see that the above simplifies to $\gamma^\intercal x + \nu > 0$ and this is $\gamma_1 + \nu > 0$ if we use $x = (1, 0)$

XOR - More parameters, same problems

- If we do this for other data points, we get, in summary:
  - $\gamma_1 + \nu > 0$; this is $y = 2$ and $x = (1, 0)$
  - $\gamma_2 + \nu > 0$; this is from $y = 2$ and $x = (0, 1)$
  - $\gamma_1 + \gamma_2 + \nu < 0$; this is from $y = 1$ and $x = (1, 1)$
  - $\nu < 0$; this is from $y = 1$ and $x = (0, 0)$
- Adding for $y = 2$ and using the case $y = 1, x = (0, 0)$, we get

$$\gamma_1 + \gamma_2 + \nu > -\nu > 0$$

- However, from $y = 1$ and $x = (1, 1)$ we had

$$\gamma_1 + \gamma_2 + \nu < 0$$

- So, we have a contradiction even in the Softmax case; we cannot find a linear classifier that works!
- Moreover, you can even try $z = \Theta\Phi x + \beta$ where $\Theta \in \mathbb{R}^{2\times d}$ and $\Phi \in \mathbb{R}^{d\times 2}$ with $d > 2$ ... A model linear in parameters will not work!

Outline

- Softmax and Cross-Entropy
- The XOR Problem
- Feed Forward Neural Networks
- Forward and Back Propagation
- Solving the XOR Problem

Generalizing Logistic regression

- We saw in Logistic Regression that for (x, y) with $x \in \mathbb{R}^2$ we have a data flow as follows
  - $z = \beta x + \alpha$
  - $a = \sigma(z)$
  - $\ell = y \log a + (1 - y) \log 1 - a$
- We get to ℓ, the log-likelihood after one linear map and one σ mapping
- Can we generalize this and add complexity to the model?

Back To Logistic Regression

- To this end suppose we have (x, y) with $x \in \mathbb{R}^d$ and we want to predict $y \in \{0, 1\}$ from x but we want more expressivity in the model
- Denote by $z^{[l]}$ a linear transformation going from "layer" $l-1$ to "layer" l
  - Usually $l = 0$ is the input layer, and $x = a^{[0]}$
  - $z^{[l]} = \beta^{[l]} a^{[l-1]} + \alpha^{[l]}$ for $l = 1, \ldots, L$
  - Note that $\beta^{[l]}, \alpha^{[l]}$ are matrices and vectors here
  - For example, $\beta^{[l]} \in \mathbb{R}^{d_l \times d_{l-1}}, \alpha^{[l]} \in \mathbb{R}^{d_l}$
- How do we go from $z^{[l]}$ to $z^{[l+1]}$? We need to define $a^{[l]}$ since

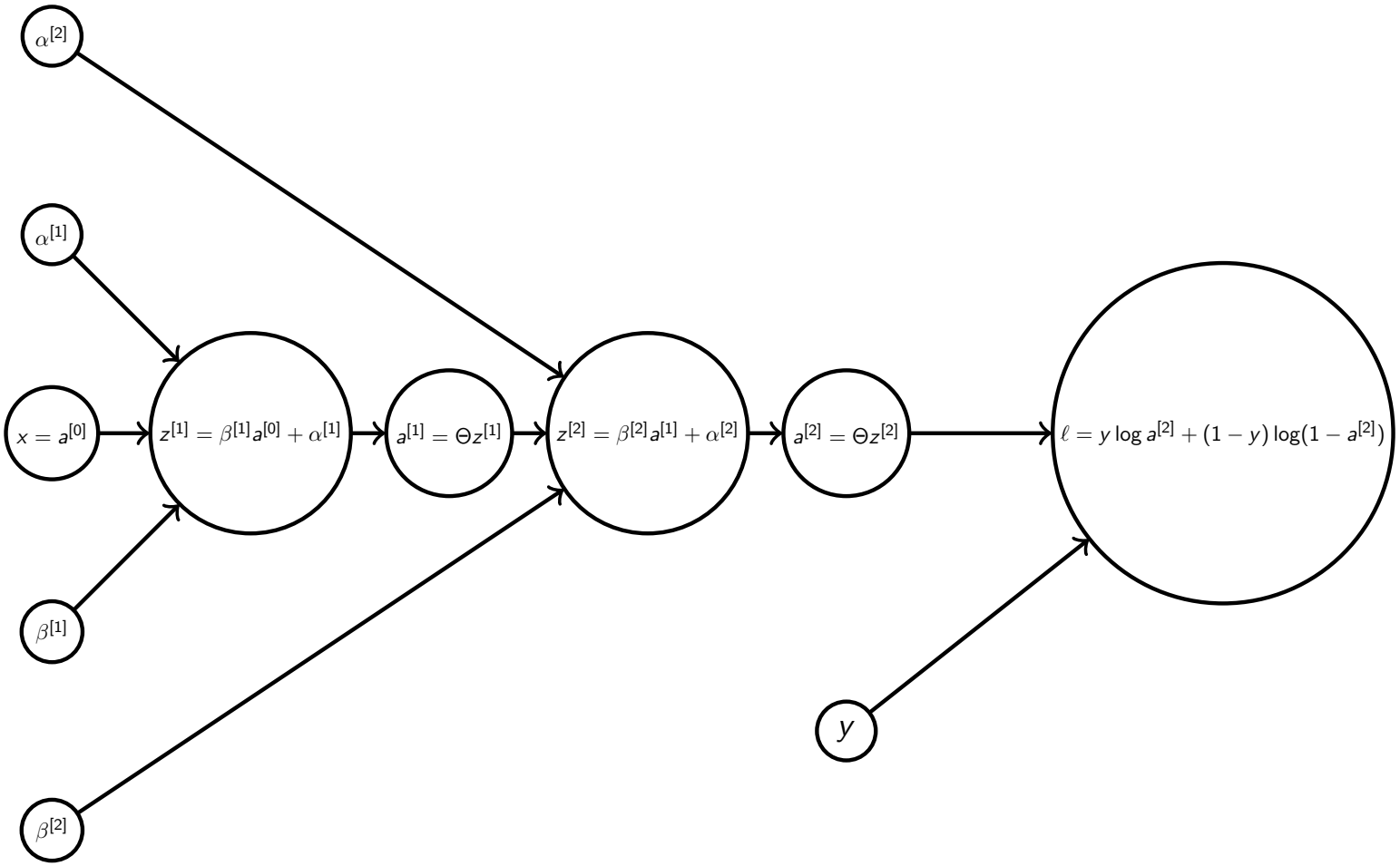
$$z^{[l+1]} = \beta^{[l+1]} a^{[l]} + \alpha^{[l+1]}$$

  and $z^{[l]} \to a^{[l]}$
- The path of transformations is

$$a^{[0]} \to z^{[1]} \to a^{[1]} \to \ldots \to z^{[L]} \to a^{[L]}$$

- You can show, if $a^{[l]} = \Theta z^{[l]}$, we basically get Logistic Regression again ... We need nonlinearity!

A Longer Computational Graph, but still Logistic Regression



Back To Logistic Regression

- Let $a^{[l]}$ be the nonlinear transformation of a layer l, so that $a^{[l]} = f^{[l]}(z^{[l]})$ where $f^{[l]}$ is a nonlinearity at layer l
  - Typically, $f^{[l]}$ is the same thing for any layer l
  - Examples of $f^{[l]}$ are
    - $f(z) = \sigma(z)$
    - $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = tanh(z)$
    - $f(z) = ReLU(z) = \max(z, 0) = z^+$
  - For a vector z, $f(z)$ means we apply f to each component of f independently
  - Each of these functions has their own properties, as we will see

In summary

- $x = a^{[0]}$
- $a^{[l-1]} \to z^{[l]}$ via a linear map
- $z^{[l]} \to a^{[l]}$ via a nonlinear function application
- ℓ, log-likelihood we want to maximize, is a (negative) Cross-Entropy between y and $a^{[L]}$

A simple Neural Network

- As a concrete example suppose we have $L = 2$ and we use $f(z) = ReLU(z)$ always
- Then the recursions go
  - $a^{[0]} = x$
  - $z^{[1]} = \beta^{[1]} a^{[0]} + \alpha^{[1]}$
  - $a^{[1]} = ReLU(z^{[1]})$
  - $z^{[2]} = \beta^{[2]} a^{[1]} + \alpha^{[2]}$
  - $a^{[2]} = \sigma(z^{[2]})$
  - $\ell = y \log a^{[2]} + (1 - y) \log 1 - a^{[2]}$
- For $L = 2$ we have 2 sets of parameters, $(\beta^{[1]}, \alpha^{[1]})$ and $(\beta^{[2]}, \alpha^{[2]})$
- Note that in this example $\beta^{[2]} \in \mathbb{R}^{1 \times d_{l-1}}, \alpha^{[2]} \in \mathbb{R}$ so that $a^{[2]} \in \mathbb{R}$

Logistic Regression and NN

- Note that Logistic Regression can be views as a "1-layer" Neural Network
- The big step between the Logistic model and the Neural Network is the additional nonlinearity introduced by the *ReLU* application
- This application, by the way, makes the model nonconvex: it is a much harder to find

$$\max_{\beta,\alpha} \ell(\beta, \alpha)$$

  - In the 90s, people did not want to do this because you can't prove Convergence Theorems with the above
  - Eventually, some Theorems ended up being proven and it did not matter since performance increased dramatically
- A model like this is called a feed forward multilayer perceptron (MLP), and it is the natural generalization of Logistic Regression

Outline

- Softmax and Cross-Entropy
- The XOR Problem
- Feed Forward Neural Networks
- Forward and Back Propagation
- Solving the XOR Problem

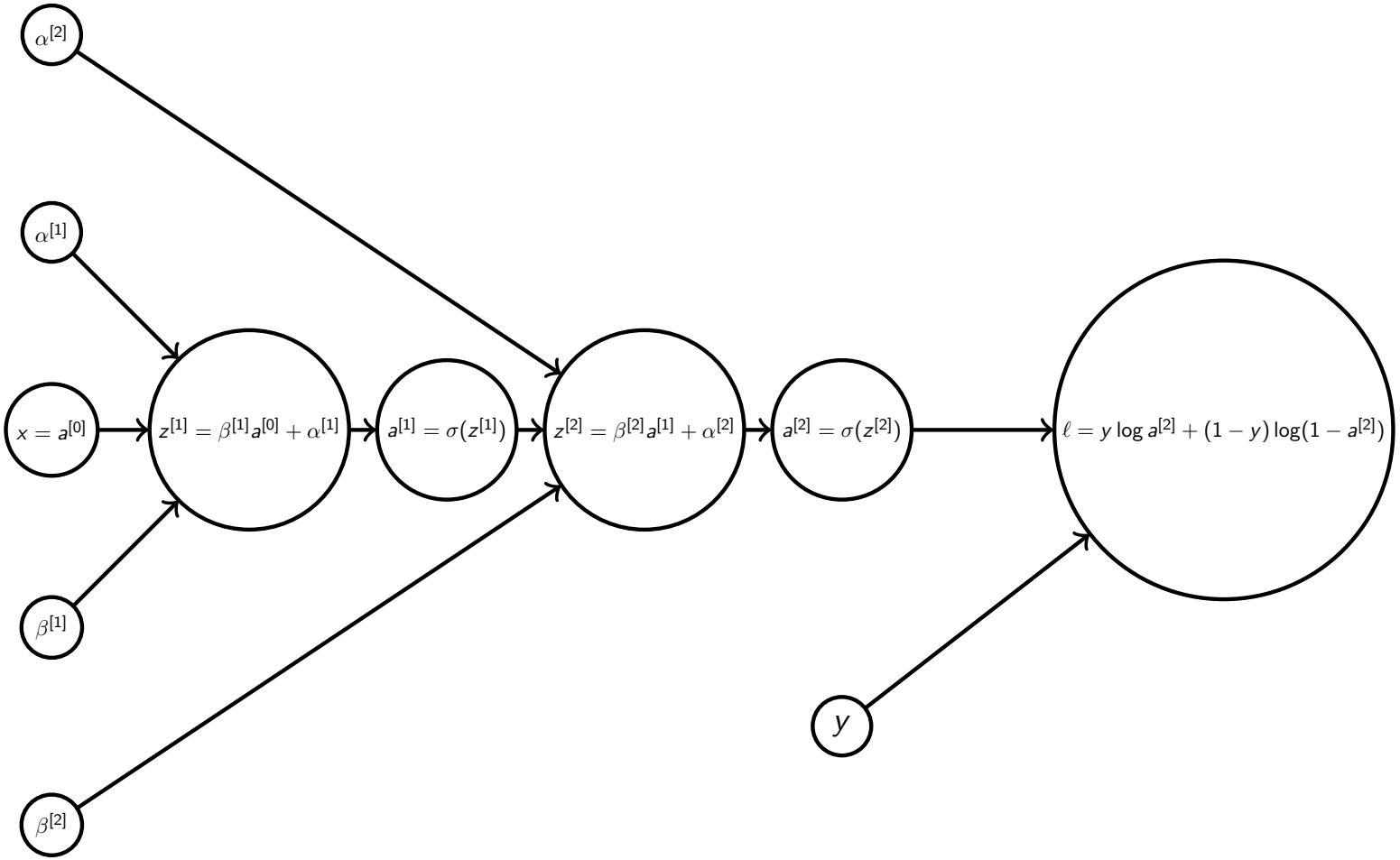
Optimization for a MLP

- How do we optimize a MLP?
- We want to solve $\max_{\{\beta^{[l]}\}_{l=1}^{L}, \{\alpha^{[l]}\}_{l=1}^{L}} \ell(\{\beta^{[l]}\}_{l=1}^{L}, \{\alpha^{[l]}\}_{l=1}^{L})$
- For the sake of example let's suppose $L = 2$ and the all $\{\beta^{[l]}\}_{l=1}^{L}$, $\{\alpha^{[l]}\}_{l=1}^{L}$, x are scalars
- For gradient ascent, we need

$$\theta = \theta + \rho \frac{d\ell}{d\theta}$$

  for the parameters $\theta = \{\beta^{[l]}\}_{l=1}^{2}, \{\alpha^{[l]}\}_{l=1}^{2}$
- Recall the flow of data could be as below
  - $a^{[0]} = x$
  - $z^{[1]} = \beta^{[1]} a^{[0]} + \alpha^{[1]}$
  - $a^{[1]} = \sigma(z^{[1]})$ (Use σ nonlinearity, not ReLU)
  - $z^{[2]} = \beta^{[2]} a^{[1]} + \alpha^{[2]}$
  - $a^{[2]} = \sigma(z^{[2]})$
  - $\ell = y \log a^{[2]} + (1 - y) \log 1 - a^{[2]}$
- What is the computational graph?

MLP Computational Graph



Getting Derivatives

- How do we get some gradients? We again consider paths and do computations
- For example, $\frac{d\ell}{d\beta^{[2]}}$
  - $\beta^{[2]} \to z^{[2]} \to a^{[2]} \to \ell$
  - $\frac{d\ell}{d\beta^{[2]}} = \frac{d\ell}{da^{[2]}} \frac{da^{[2]}}{dz^{[2]}} \frac{dz^{[2]}}{d\beta^{[2]}}$
  - This is $\frac{y-a^{[2]}}{a^{[2]}(1-a^{[2]})}(a^{[2]}(1-a^{[2]}))a^{[1]} = (y-a^{[2]})a^{[1]}$
- Similarly, $\frac{d\ell}{d\alpha^{[1]}}$
  - $\alpha^{[1]} \to z^{[1]} \to a^{[1]} \to z^{[2]} \to a^{[2]} \to \ell$
  - $\frac{d\ell}{d\alpha^{[1]}} = \frac{d\ell}{da^{[2]}} \frac{da^{[2]}}{dz^{[2]}} \frac{dz^{[2]}}{da^{[1]}} \frac{da^{[1]}}{dz^{[1]}} \frac{dz^{[1]}}{d\alpha^{[1]}}$
  - This is
$$(y-a^{[2]})\beta^{[2]}(a^{[1]}(1-a^{[1]}))$$
- So, the gradients can be computed analytically

Derivatives of Nonlinearities

- What if we used $a^{[1]} = ReLU(z^{[1]}) = \max\{z^{[1]}, 0\}$ or $a^{[1]} = tanh(z^{[1]})$ in the above?
- For $f(z) = ReLU(z)$ you can show that
  - The gradient is 1 if $z > 0$
  - 0 otherwise
  - Succinctly this can be written as the indicator function $\frac{dReLU}{dz} = 1_{(0,\infty)}(z)$
  - Here $1_A(z)$ is 1 if $z \in A$ and zero otherwise
  - Technically, this is not a gradient, it is a sub-gradient, but this does not matter: pretty much everything "just works"
- For $f(z) = tanh(z)$ you can show that
  - $\frac{dtanh}{dz} = 1 - tanh^2(z)$
- For $f(z) = \sigma(z)$ we know that
  - $\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$

Why ReLU is better than σ

- What happens to the derivatives of $\sigma(z)$ when z is large (∞)? What if z is small (-∞)?
- We have $\sigma(z) \sim 1$ for z very large and $\sigma(z) \sim 0$ for z very small
- But we also have $\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$
- What this means is that for these extreme values, $\frac{d\sigma}{dz} = 0$ and this implies that if $z^{[1]}$ is extreme then

$$\frac{d\ell}{d\alpha^{[1]}} = \frac{d\ell}{da^{[2]}} \frac{da^{[2]}}{dz^{[2]}} \frac{dz^{[2]}}{da^{[1]}} \frac{da^{[1]}}{dz^{[1]}} \frac{dz^{[1]}}{d\alpha^{[1]}} = 0$$

- This is a problem: we will make no progress on this parameter when we do gradient ascent
  - The update rule is $\theta_{new} = \theta_{old} + \rho \frac{d\ell}{d\theta} = \theta_{old}$ in this case
- This type of issue, where you get zero gradients and don't learn, is usually called the "saturation" or "vanishing gradient" problem for Deep Neural Networks

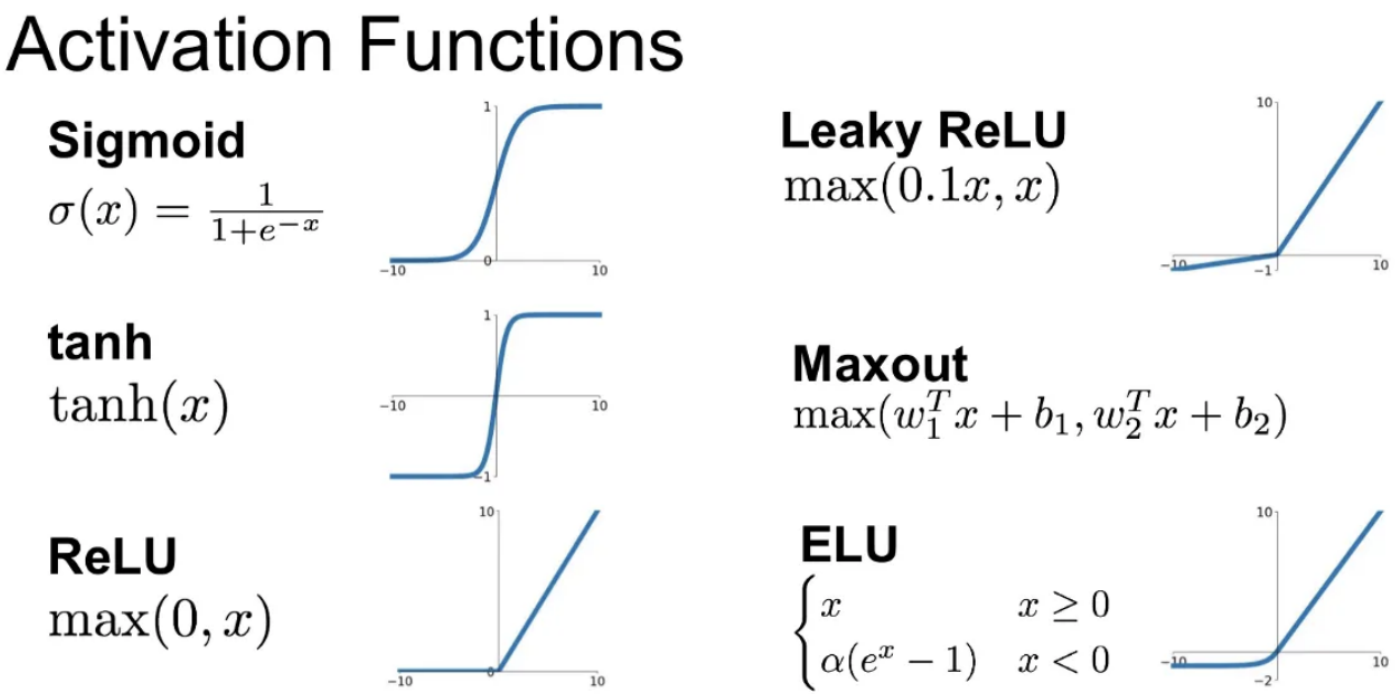
Why ReLU is better than *tanh*

- What happens to the derivatives of $\tanh(z)$ when z is large (∞)? What if z is small (-∞)?
- We have $tanh(z) \sim 1$ for z very large and also $tanh(z) \sim -1$ for z very small
- Now we have $\frac{dtanh}{dz} = 1 - tanh^2(z)$ so for extreme values we get a 0 gradient and the same problem happens as for σ
- We'll see some methods later which aim to fix this
- $ReLU$ and (for $\gamma > 0$ fixed)

$$Leaky - ReLU_\gamma(z) = z1_{[0,\infty)}(z) - \gamma z1_{(-\infty,0)}(z)$$

  solve this
- $Leaky - ReLU$ specifically always has a gradient that's nonzero: 1 if $z > 0$ and $-\gamma < 0$ otherwise

Visually looking at Activations



Different Activation Functions and their Graphs

Outline

- Softmax and Cross-Entropy
- The XOR Problem
- Feed Forward Neural Networks
- Forward and Back Propagation
- Solving the XOR Problem

Solving the XOR Problem

- Consider two variables x_1, x_2 that take on values of 1 or 0
- Set $y = x_1 + x_2 - 2x_1x_2 + 1$
- The y is known as XOR and the values are as follows
  - If $x_1 = 1$ and $x_2 = 1$ then $y = 1$
  - If $x_1 = 1$ and $x_2 = 0$ then $y = 2$
  - If $x_1 = 0$ and $x_2 = 1$ then $y = 2$
  - If $x_1 = 0$ and $x_2 = 0$ then $y = 1$
- Problem: Given 4 data points $\{(x^{(i)}, y^{(i)})\}_{i=1}^{4}$ that represent the 4 points above, can we find a classifier that can correctly classify these points?
- We want
$$p(y = 1|x_1 = 1, x_2 = 1) > 1/2$$
- We want
$$p(y = 2|x_1 = 1, x_2 = 0) > 1/2$$
- And similarly for the other two cases ...

Solving the XOR Problem

- Let's consider a slightly more complicated model and see if we can get anywhere
- In particular, set $z^{[1]} = (\beta^{[1]})^\intercal x + \alpha^{[1]}$ as before
- Here, we first map x to another vector of dimension 2 via $\beta^{[1]} \in \mathbb{R}^{2\times 2}$ and $\alpha^{[1]} \in \mathbb{R}^{2\times 1}$
- Set $\beta^{[1]}$ to

$$\beta^{[1]} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

  and $\alpha^{[1]} = (0, -1)$
- Set $a^{[1]} = ReLU(z^{[1]}) = \max(z^{[1]}, 0) = \max((\beta^{[1]})^\intercal x + \alpha^{[1]}, 0)$

XOR - Non Linearities are Critical

- This is

$$ReLU((x_1+x_2, x_1+x_2-1)) = (ReLU(x_1+x_2), ReLU(x_1+x_2-1))$$

  - If $x = (1,0)$ then $a^{[1]} = max((1,0),0) = (1,0)$
  - If $x = (0,1)$ then $a^{[1]} = max((1,0),0) = (1,0)$
  - If $x = (1,1)$ then $a^{[1]} = max((2,1),0) = (2,1)$
  - If $x = (0,0)$ then $a^{[1]} = max((0,-1),0) = (0,0)$
- Let $\beta^{[2]}$ and $\alpha^{[2]}$ be another set of parameters before the Softmax
- $z^{[2]} = \beta^{[2]} a^{[1]} + \alpha^{[2]}$
- We consider $a^{[2]} = softmax(z^{[2]})$
- Now set $p(y|x) = \frac{e^{(\beta_y^{[2]})^\intercal a^{[1]} + \alpha_y^{[2]}}}{\sum_{y'} e^{(\beta_{y'}^{[2]})^\intercal a^{[1]} + \alpha_{y'}^{[2]}}}$
  - Here, we have $\beta^{[2]} \in \mathbb{R}^{2\times 2}$ and $\alpha^{[2]} \in \mathbb{R}^2$
  - These are parameters we apply to a before the Softmax
- We want $p(y^{(i)}|x^{(i)}) > 1/2$, as before

XOR - Can we find a solutions?

- For example, we need

$$p(y=2|x) = \frac{e^{(\beta_2^{[2]})^\intercal a^{[1]} + \alpha_2^{[2]}}}{\sum_{y'} e^{(\beta_{y'}^{[2]})^\intercal a^{[1]} + \alpha_{y'}^{[2]}}} > \frac{1}{2}$$

- This means (we force factor the numerator and divide)

$$p(y=2|x) = \frac{1}{1 + e^{-[(\beta_2^{[2]} - \beta_1^{[2]})^\intercal a^{[1]} + \alpha_2^{[2]} - \alpha_1^{[2]}]}} > \frac{1}{2}$$

- On the other hand we have

$$p(y=1|x) = 1 - \frac{1}{1 + e^{-[(\beta_2^{[2]} - \beta_1^{[2]})^\intercal a^{[1]} + \alpha_2^{[2]} - \alpha_1^{[2]}]}} > \frac{1}{2}$$

XOR - Can we find a solutions?

- As with Logistic Regression, this is translated into several conditions where we set $\gamma = \beta_2^{[2]} - \beta_1^{[2]} \in \mathbb{R}^2$ and $\nu = \alpha_2^{[2]} - \alpha_1^{[2]} \in \mathbb{R}$

$$p(y = 2|x) = \frac{1}{1 + e^{-[\gamma^\intercal a^{[1]} + \nu]}} > \frac{1}{2}$$

- Doing a little algebra, this becomes
  - If $y = 2$, $\gamma^\intercal a^{[1]} + \nu > 0$
  - If $y = 1$, $\gamma^\intercal a^{[1]} + \nu < 0$
- We therefore need to find γ and ν such that
  - $\gamma^\intercal(1,0) + \nu > 0$ for $y = 2$, $x = (1,0) \rightarrow a^{[1]} = (1,0)$
  - $\gamma^\intercal(1,0) + \nu > 0$ for $y = 2$, $x = (0,1) \rightarrow a^{[1]} = (1,0)$
  - $\gamma^\intercal(0,0) + \nu < 0$ for $y = 1$, $x = (0,0) \rightarrow a^{[1]} = (0,0)$
  - $\gamma^\intercal(2,1) + \nu < 0$ for $y = 1$, $x = (1,1) \rightarrow a^{[1]} = (2,1)$

XOR - Can we find a solutions? Let's guess and check

- Set $\gamma = (1, -2)$ and $\nu = -\frac{1}{2}$, we can see that all the inequalities are satisfied
  - $(1, -2)^\intercal(1, 0) - \frac{1}{2} = 1/2 > 0$ for $y = 2$, $x = (1, 0)$
  - $(1, -2)^\intercal(1, 0) - \frac{1}{2} = 1/2 > 0$ for $y = 2$, $x = (0, 1)$
  - $(1, -2)^\intercal(0, 0) - \frac{1}{2} = -1/2 < 0$ for $y = 1$, $x = (0, 0)$
  - $(1, -2)^\intercal(2, 1) - \frac{1}{2} = -1/2 < 0$ for $y = 1$, $x = (1, 1)$
- Thus, we have infinitely many parameters that we can pick from that could make this work
- The crucial step in all this was $a^{[1]} = max(z^{[1]}, 0)$, the introduction of a nonlinearity!

References

- Andrew NG's notes on MLP
- Michael Collins' notes on NLP, XOR, and MLP